

Beyond Manual Inspection: Automated Semantic Evaluation of SAE Features for Mechanistic Interpretability

Large language models like GPT-4 or Claude process text through hundreds of layers of densely interconnected neurons, and while they produce impressively coherent outputs, understanding why they produce those outputs is surprisingly hard. One of the core reasons is a phenomenon called superposition: individual neurons in an LLM don't cleanly represent single concepts. Instead, each neuron responds to dozens of unrelated ideas simultaneously, a property called polysemanticity. This makes it very difficult to trace a model's reasoning back to anything interpretable. Mechanistic interpretability is the subfield of ML research trying to crack this problem. Mechanistic interpretability research attempts to reverse-engineer the internal computations of neural networks well enough to understand, predict, and attempt to better control their behavior. Sparse autoencoders (SAEs) have recently emerged as one of the most promising tools in this space. The core idea is to train a secondary neural network on top of an LLM's internal activations that learns to re-express them in a much higher-dimensional space where individual features are more likely to map cleanly onto human-interpretable concepts. This is akin to translating the LLM's internal language, which is dense, overlapping, and hard to read, into a sparser vocabulary where individual words are easier to isolate. For useful further reading on this topic, check out some of the seminal papers in this space linked in the "Further Reading" section.

While SAEs have emerged as one of the most promising tools in mechanistic interpretability, their utility and adoption thus far has been severely limited by a key bottleneck in their design: since they are trained directly from an LLM's latent space, they lack an interpretable ground truth that can be used to easily identify meaningful SAE features. This makes the process of identifying meaningful SAE features time-intensive and manual - the standard approach is to manually pull up activation examples for individual features and eyeball them until something clicks. This works well and can be used to demonstrate the utility of SAEs in toy examples and proofs of concept, but it doesn't scale to the types of larger scale mechanistic interpretability experiments where SAEs are truly needed.

This post introduces a data-driven alternative. Rather than hand-selecting features for inspection, this framework allows users to automatically score every feature in their SAE along two complementary axes: how semantically consistent are the contexts that activate it, and how consistent is the specific token that activates it. Together these two scores map every SAE feature onto an interpretability landscape, separating the features that cleanly separate abstract concepts like "sporting achievement" or "units of money" from concrete token-anchored ones like "the period at the end of a sentence" or "the word 'the'", and are able to distinguish both from features that are genuinely noisy or effectively dead.

Everything described here is implemented in `sae_trainer`, an open-source Python package for training and evaluating SAEs ([GitHub](#)). You can explore the full feature landscape for the example SAE interactively [here](#).

Background:

Large language models (LLMs) are powerful and ubiquitous tool in everyday life, but while they often provide useful answers to questions, backtracking through their reasoning is a

notoriously difficult task. One of the main reasons for this is as their internal representations tend to be polysemantic — individual neurons respond to multiple, semantically unrelated concepts simultaneously [[Toy Models of Superposition](#)]. Sparse autoencoders (SAEs) have emerged as a promising tool to help open the black box of LLM reasoning by learning an overcomplete, sparse decomposition of an LLM's residual stream activations into a higher-dimensional latent space in which individual features are more likely to be monosemantic [[Towards Monosemanticity: Decomposing Language Models With Dictionary Learning](#)]. This decomposition has shown considerable promise: recent work has demonstrated that SAE features can be meaningfully interpreted, steered, and causally linked to model behavior [[Scaling and evaluating sparse autoencoders](#)]. However, a key bottleneck limiting the broader adoption of SAEs is the absence of an interpretable ground truth for SAE features — determining whether a given feature is semantically meaningful currently requires significant manual inspection, typically via labor-intensive analysis of the token distributions that maximally activate each feature. This project seeks to address this limitation by introducing a data-driven framework for automatically identifying and evaluating semantically meaningful SAE features, reducing reliance on manual intervention, and packaging this approach into a re-usable python package to help interpretability researchers identify meaningful SAE features in their own work. The package is fully open source with the goal of lowering the barrier to entry for mechanistic interpretability research, and the experiments described here are intended to serve as a reproducible baseline for future work in this area.

For these experiments, a TopK-SAE (k=40) targeting layer 12 of a Qwen-0.5B LLM was trained for 50 epochs on the OpenWebText dataset using `sae_trainer`. The goal here isn't to find the optimal SAE configuration, but to demonstrate what the semantic evaluation tools can reveal even with a straightforward proof-of-concept model. A systematic hyperparameter search is left as future work and would certainly yield interesting results.

Approach:

This project introduces a suite of per-feature metrics that are computed for each SAE-latent feature and used to identify semantically interesting SAE features in a data driven way: “feature_context_specificity_score”, “feature_token_specificity_score”, and “composite_score”. I will describe each in the order in which they were introduced, starting with feature_context_specificity score.

To generate feature_context_specificity_score for each SAE feature, SAE activations are first collected per input token using a dictionary-learning feature tracing method similar to the one described here [[Towards Monosemanticity: Decomposing Language Models With Dictionary Learning](#)]. To summarize, these activations are collected by first hooking the target layer in the LLM that corresponds to the layer the SAE was trained on and saving all the LLM-activations that are generated at that layer for each output token in the eval set (prompt tokens are excluded by default). These LLM-activations are then passed through the SAE, and the feature_id and activation_value are recorded for each of the top-k SAE features that fire on each token, conditioned on the activation_value being above some min_activation. Features are recorded only if they fire on at least min_prompts (default: 5) distinct prompts to prevent tracking features that fire only on a few prompts, since these features likely lack any substantial semantic interpretability (otherwise they would have fired on more prompts!). This gives a flat

table of token-level firing events across all prompts in the eval set, with SAE activation value recorded for each firing event.

The next step in computing `feature_context_specificity_score` is to look for similarities among tokens in each SAE-feature’s firing profile. This is done by utilizing a pre-trained Sentence Transformer model (all-MiniLM-L6-v2) to embed small context windows (+/- 8 tokens) around each token in the token-level firing table. These embeddings are L2-normalized so that they can be readily compared, and saved along with the token-level firing table. They are then used to evaluate the semantic similarity of context windows that lead to SAE-feature activation for each SAE feature. This is done by computing `feature_context_specificity_score`, the weighted mean pairwise cosine similarity of the context embeddings for each SAE-feature where the weights are the activation values for that SAE feature on the token corresponding to that embedding vector (normalized to sum to 1 to ensure that high-activating contexts contribute more to the `feature_context_specificity_score`). Because SAE-features can fire on multiple tokens in the same prompt, and these tokens will tend to have much higher pairwise similarity in their embedding vectors than tokens from distinct prompts, especially if their context windows partially overlap, cross-prompt masking is applied so that `feature_context_specificity_score` is only computed between embedding vectors drawn from tokens from distinct prompts. This means that a high `feature_context_specificity_score` indicates that a SAE-feature is firing on semantically similar contexts across different prompts.

`Feature_context_specificity_scores` allow for SAE features to be rank-ordered based on the semantic similarity of their activating contexts, but they do not inherently provide any sense of how much more coherent each feature’s activating contexts are than the baseline similarity of random context windows drawn from the eval set. To provide this semantic grounding, “`specificity_vs_baseline`” is computed by first drawing N (default: 1000) random context pairs from the full pool of embeddings across all features to compute a “`baseline_specificity_score`”, then subtracting each this value from each feature’s “`feature_context_specificity_score`”. This gives us a sense of both how different SAE-features stack up against each other, and against a random baseline, giving us a robust picture of the feature specificity tied to each SAE-feature.

Analysis of features with high `feature_context_specificity_scores`:

A high `feature_context_specificity_score` means that a feature only fires on semantically similar concepts across all the prompts that activate it. In theory, manual analysis of the top activating contexts of these features should reveal this semantic similarity. Looking at the top 3 activating contexts for the top 5 features ranked by `feature_context_specificity_score`, this is exactly what we see.

Feature Index	Feature Card	Feature Summary
---------------	--------------	-----------------

6899	Feature #6899 Hits: 26 Mean act: 3.233 Max act: 5.246 Prompts: 10 Top Activating Contexts [pos=196, act=5.246] ... five-minute penalty, Brian Wilsie scored the game's lone goal with a shot... [pos=190, act=3.938] ... hand in the goal. The ball went in! And we were... [pos=191, act=3.594] ... and possibly the Coleman Medal with his fine finishing. It was a tough week for...	Goal-scoring actions and outcomes in sports contexts.
5461	Feature #5461 Hits: 68 Mean act: 4.098 Max act: 8.024 Prompts: 23 Top Activating Contexts [pos=213, act=8.024] ...arris) March 25, 2015 He's the first cousin... [pos=215, act=7.489] ... Sept. 18, 201... [pos=221, act=7.317] ... April 30, 2015 In the meantime,...	The opening digit(s) of a year within a fully specified date.
3554	Feature #3554 Hits: 43 Mean act: 3.844 Max act: 6.573 Prompts: 20 Top Activating Contexts [pos=282, act=6.573] ... posted by Nesta on March 4, ... [pos=211, act=6.457] ... homeopathic treatment. But on July 1, 2015, the... [pos=210, act=6.083] ... before Newtown Local Court on July 2. The police have not...	The opening digit(s) of a day within a fully specified date.
454	Feature #454 Hits: 23 Mean act: 3.123 Max act: 5.620 Prompts: 12 Top Activating Contexts [pos=247, act=5.620] ...00 - 17:00 (CET... [pos=183, act=5.113] ... next thing I know it is 3 pm," he says. He has been... [pos=201, act=4.526] The launch should take place in the morning, and the spacecraft will...	Time-of-day expressions, spanning clock notation and period of day terms.
6315	Feature #6315 Hits: 103 Mean act: 3.775 Max act: 7.279 Prompts: 37 Top Activating Contexts [pos=201, act=7.279] India will spend an estimated \$6 billion on its military in 2... [pos=214, act=6.936] ... announced the city would spend \$15 million to renov... [pos=201, act=6.731] ... and Associates includes a \$1.4 million cost to rebuild the bridge, with the...	Large monetary denomination words (millions/billions) following a dollar figure.

Analysis of features with low feature_context_specificity_scores - introduction of the feature_token_specificity_score:

While a high feature_context_specificity_score is a good indicator that a feature has some semantic meaning, a low feature_context_specificity_score can have multiple different interpretations, not all of which indicate that a feature is not semantically meaningful. This can be seen by looking at the top 3 activating contexts for the bottom three features ranked by feature_context_specificity_score:

Feature Index	Feature Card
2304	Feature #2304 Hits: 40 Mean act: 3.273 Max act: 6.395 Prompts: 32 Top Activating Contexts (pos=176, act=6.395) ... Americans – two-thirds of whom are overweight or obese – do not exercise at all... (pos=192, act=6.381) ... of us who can't afford them, or who simply don't want to... (pos=176, act=6.355) ... Unity gives the option to use C# or VB.NET, and you can...
1504	Feature #1508 Hits: 67 Mean act: 2.345 Max act: 3.389 Prompts: 63 Top Activating Contexts (pos=197, act=3.389) ... (Shark) comes with natural leather bolster, which is very comfortable to use... (pos=191, act=3.388) ... of rainwater as radioactive isotopes spread throughout the country. The agency said it is... (pos=287, act=3.129) ... the movie's opening credits, as Gone with the Wind). The film was released in...
1784	Feature #1784 Hits: 1687 Mean act: 2.852 Max act: 6.042 Prompts: 386 Top Activating Contexts (pos=211, act=6.042) ... The large built-in cabinets and the large windows make the apartment very bright... (pos=322, act=5.995) ... height of 10 meters and a diameter of 45 centimetres (...) (pos=174, act=5.829) ... to 100 and is the stability of the disc when thrown at its intended...

Features 1504 and 1784 do not have any clear semantic similarity in their activating contexts, but feature 2304 is clearly activating on the word “or” in each of these contexts. Expanding its feature card to show the top 10 activating contexts shows that this pattern continues beyond the top 3 activating contexts:

Feature #2304

Hits: 40 Mean act: 3.273 Max act: 6.395 Prompts: 32

Top Activating Contexts

[pos=176, act=6.395] ... Americans – two-thirds of whom are overweight **or** obese – do not exercise at all...

[pos=192, act=6.381] ... of us who can't afford them, **or** who simply don't want to...

[pos=176, act=6.355] ... Unity gives the option to use C# **or** VB.NET, and you can...

[pos=187, act=6.138] ... had put the gun in a locked safe **or** in a locked container...

[pos=176, act=6.061] ... that data to either steal your information, **or** to create a false identity...

[pos=179, act=6.002] ... 'Tell Hugh Grant to shut up now **or** he will be killed by...

[pos=188, act=5.864] ... allegations, a particular defendant may have directly **or** indirectly downloaded and/or shared the copyrighted material...

[pos=222, act=5.817] ... Jim, and Vivre une vie. **Or**, if you're a fan...

[pos=226, act=5.772] ... 10 hours of battery life, **or** 350 minutes...

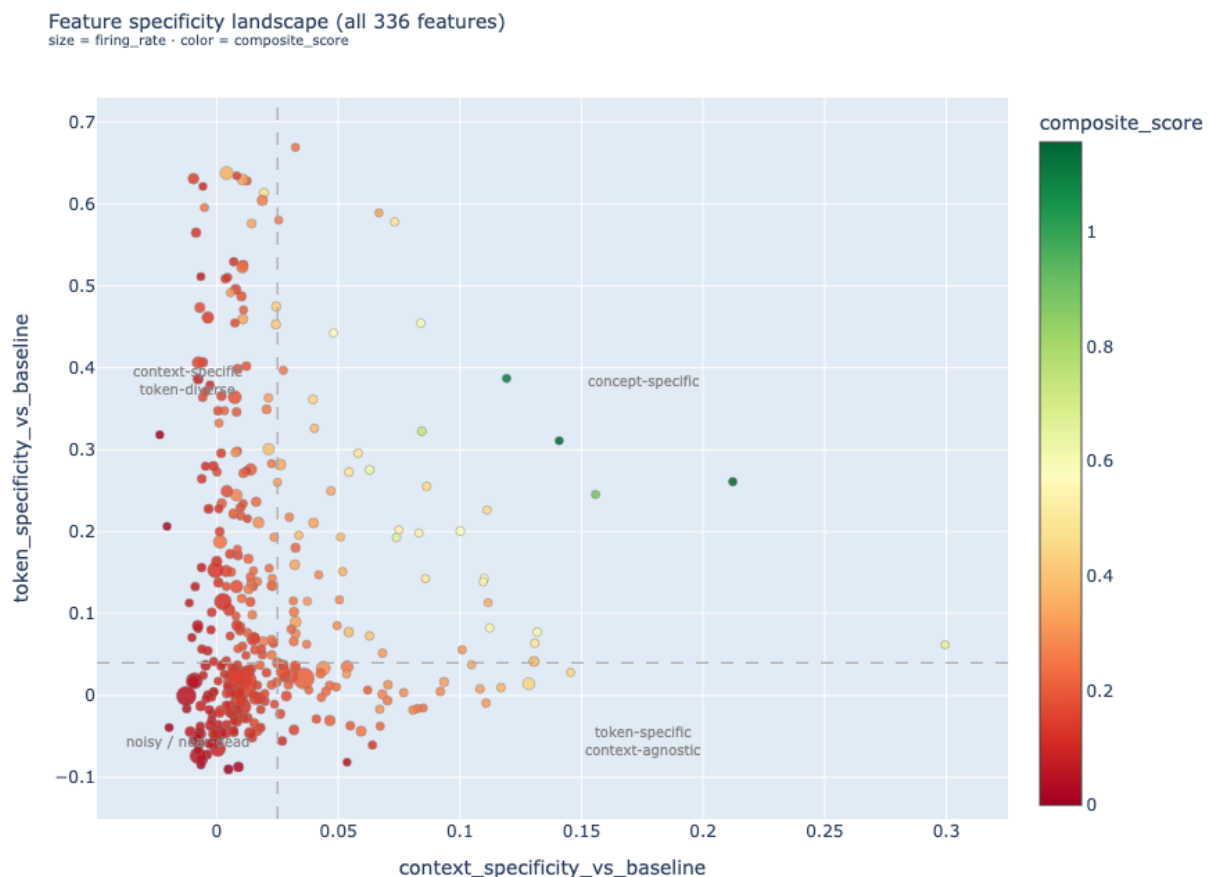
[pos=184, act=5.614] ... will either be a fresh cow manure **or** a manure that...

Feature 2304 is clearly a meaningful feature, but because `feature_context_specificity_score` uses the embedding of a context window around a given activating token to quantify semantic similarity, it fails to capture the similarity of the actual token that is causing activation. To account for this and capture similar features that are clearly semantically meaningful at the token level, we'll introduce another metric, `feature_token_specificity_score` which is essentially a special case of `feature_similarity_score` with `context_window` set to 0 (ie. it computes the `feature_similarity_score` based solely on the activating token, not its context). This provides two complimentary axes on which feature interpretability can be ranked. The table below describes the interpretation of `feature_context_specificity_score` vs. `feature_token_specificity_score`

Context- vs. Token-specificity definitions

Context specificity	Token Specificity	Interpretation
High	High	Fires on specific words in specific contexts - most interpretable
High	Low	Fires diverse token set in specific contexts - semantic/conceptual features
Low	High	Fires on specific tokens independent of context - token-anchored features
Low	Low	Less clear - could be genuinely noisy / polysemantic features

These two axes, `feature_context_specificity_score` and `feature_token_specificity_score`, provide a useful and visually appealing means of plotting SAE features to indicate their semantic utility. In an attempt to simultaneously capture a feature's utility along both of these axes, a third per-feature metric, “`composite_score`”, is introduced. `Composite_score` is defined as the harmonic mean of `feature_context_specificity_score` and `feature_token_specificity_score`, scaled by the `mean_activation` level to account for features that activate more strongly than those that activate weakly. The harmonic mean is used here because it penalizes features that score well on one axis but poorly on the other. The full feature profile along both of these axes for our test SAE can be seen in the figure below, where each feature is colored according to its composite score. This figure can also be viewed interactively in the Github pages demo here: [link](#))

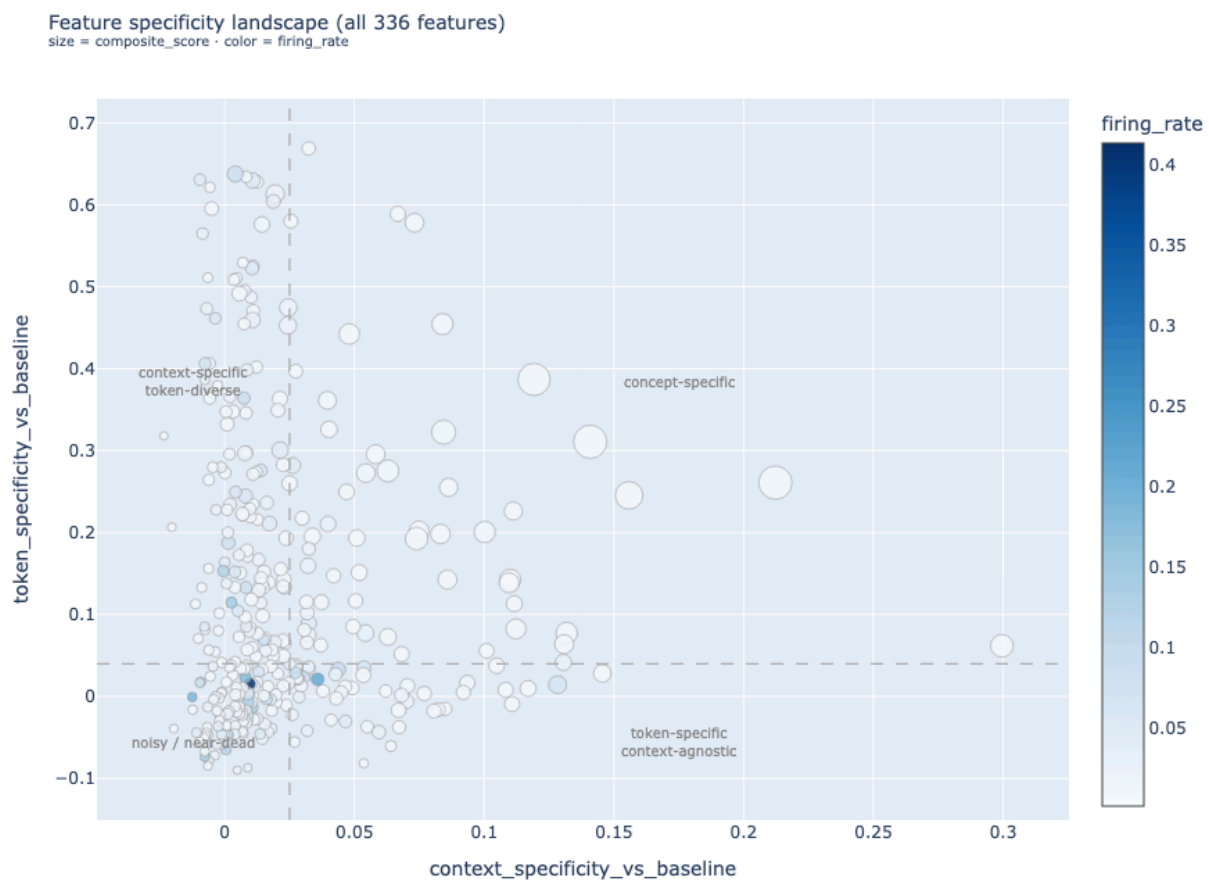


Full feature landscape. Feature interpretability cleanly increases as you move upward / rightward on this plot.

Noisy vs. pseudo-dead features

There is another potential confounder that has been only partially considered up to this point, one that is particularly impactful in the low-context / low-token specificity regime in which

many of these features exist: features can have low context- and token-level specificity scores because they either fire too often on un-related tokens (“noisy features”) or fire too infrequently altogether (“dead features”). Fully “dead features” (ie. features in the SAE that rarely activate, if at all, in the prompt set) have already been accounted for via “min_prompts”, a useful configuration parameter that allows us to filter features that activate on fewer than “min_prompts” from the analysis entirely. But there is still a useful distinction to be made between features that are low on both context- and token-level specificity axes because they fire very infrequently on semantically distinct contexts (“pseudo-dead features”) and those that fire often on semantically distinct contexts (“noisy features”). To visualize this distinction, we can actually utilize the same metrics as above, with slightly different visualization parameters:



Full feature landscape, colored by firing rate to highlight “noisy” features vs. “pseudo-dead” features.

This visualization highlights a clear distinction between features in the low context- and token-level specificity score regime. Certain features (notably 3229, 3661, 1784, which conveniently correspond to the largest feature bubbles in the lower left quadrant in the linked demo) are low on both specificity score axes, but have very high firing rates (“noisy features”). Most of the others in this regime have a relatively low firing rate (“pseudo-dead features”).

Spot-checking the top activating contexts for both groups confirms what the scores suggest: neither shows any obvious semantic consistency, as shown in the table below.

Feature Index	Feature Classification (noisy vs. pseudo-dead)	Feature Card
3229	noisy	Feature #3229 Hits: 3716 Mean act: 2.935 Max act: 6.404 Prompts: 756 Top Activating Contexts [pos=172, act=6.404] ... the early stages of implementing a federal consent decree that requires police departments to work with the... [pos=288, act=6.288] ..., who is seeking urgent financial and technical assistance to help refugees find shelter and food... [pos=193, act=6.272] ... and money laundering. The new guidelines also require that banks hire employees...
726	pseudo-dead	Feature #726 Hits: 173 Mean act: 2.792 Max act: 5.165 Prompts: 151 Top Activating Contexts [pos=188, act=5.165] ... the mask, but the man was so animated that... [pos=175, act=4.674] ... which women become fans is more complicated and nuanced than we had previously thought. We spoke... [pos=187, act=4.491] ... the incident: "We have been working diligently to address...
1784	noisy	Feature #1784 Hits: 1687 Mean act: 2.852 Max act: 6.042 Prompts: 386 Top Activating Contexts [pos=211, act=6.042] ... The large built-in cabinets and the large windows make the apartment very bright... [pos=322, act=5.995] ... height of 10 meters and diameter of 45 centimetres (...) [pos=174, act=5.829] ... to 100 and is the stability of the disc when thrown at its intended...
3030	pseudo-dead	Feature #3030 Hits: 312 Mean act: 3.248 Max act: 7.677 Prompts: 92 Top Activating Contexts [pos=244, act=7.677] ... Francisco's SoMa neighborhood on forgery warrants and suspicion of felony burglary. Image... [pos=280, act=7.645] ... will adjust to the funding capitalization goals set by the project... [pos=245, act=7.814] ... 37 Gt CO 2 to @...

While the distinction between noisy and pseudo-dead features is worth exploring, it's worth mentioning here that SAE used in these experiments is intentionally a modest proof-of-concept, so a large population of both is expected. A cleaner experiment would be to apply these same

evaluation tools to a state-of-the-art SAE trained at scale on a frontier model, where I'd expect the bulk of features to push toward the upper-right quadrant, making whatever remains in the low-specificity regime a much more meaningful signal.

Conclusion:

This work was intended to help address a weakness in the mechanistic interpretability field - the lack of a reliable, data driven approach for evaluating interpretability using SAEs. There are two potential research directions this can lead, both equally useful to the field. The first is more of a continuation of this line of research, to continue improving the tools available to us for evaluating SAE interpretability. One example that comes to mind is the reliance on a fixed context window size for calculating `feature_context_specificity_scores` throughout this experiment. It is possible (even likely) that context width is an important variable in determining whether or not an SAE feature will activate. Maybe some features that seem to be “pseudo-dead” based on the fixed context window size used here are actually attending to concepts at a wider context window than our `feature_context_specificity_scores` are able to capture. Future work in this vein can ask questions like this to classify more activation profiles to inject interpretability into more of these features.

A divergent but equally useful line of work is to use the tools developed here to dive deeper into the mechanistic interpretability of LLMs. Rather than needing to manually select SAE features for analysis, this framework enables systematic, automated identification of the features most causally relevant to a given context or task. For example, this tool can be used to identify which SAE features are most active when an LLM reasons about code versus natural language, or how the active feature landscape shifts between factual question-answering and open-ended generation. This opens the door to a richer, data-driven approach to SAE-based circuit analysis: rather than beginning with a hypothesis about which features matter and verifying it manually, researchers can use specificity scoring to surface candidate features automatically and then interrogate their causal roles through targeted activation steering or ablation experiments. This will hopefully enable richer and more scalable SAE feature selection, and allow us to ask deeper questions about the internal mechanisms of state-of-the-art LLMs, ultimately moving the field closer to a systematic, rather than anecdotal, understanding of how these models represent and process information.

Further Reading

[Toy Models of Superposition](#) — Elhage et al., Anthropic (2022)

The foundational paper for understanding why polysematicity exists in the first place. If the motivation for SAEs is new to you, start here.

[Towards Monosemanticity](#) — Bricken et al., Anthropic (2023)

The paper that established SAEs as the go-to tool for mechanistic interpretability. Introduces the manual feature inspection workflow that this post aims to automate.

[Sparse Autoencoders Find Highly Interpretable Features in Language Models](#) — Cunningham et al. (2023)

An early parallel effort training SAEs on GPT-2, worth reading alongside Bricken et al. for a broader picture of where the field started.

[Scaling and Evaluating Sparse Autoencoders](#) — Gao et al., OpenAI (2024)

Introduces the TopK-SAE architecture used in this post, and examines how SAE quality scales with model and dictionary size.

[Sparse Feature Circuits: Discovering and Editing Interpretable Causal Graphs in Language Models](#) — Marks et al. (2024)

Introduces a method for discovering sparse circuits of causally implicated SAE features, and demonstrates a fully unsupervised interpretability pipeline. The feature tracing approach used in this post draws on ideas from this work.